

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática
Aprendizagem e Decisão Inteligentes
Ano Letivo 2025/2026

Relatório de Resultados

Análise de Dados e Modelação com KNIME

Spotify Tracks & Consumo Energético

Grupo 11

Marco Sèvegrand A106807
Francisco Martins A106902
Nuno Rebelo A106825
Marco Ferreira A106933

16 de maio de 2026

Índice

1	Introdução	1
	Parte I — Tarefa 1: Dataset de Grupo (Spotify Tracks)	2
2	Domínio e objetivos	2
2.1	Domínio analisado	2
2.2	Objetivos	3
3	Metodologia CRISP-DM	3
3.1	Compreensão dos dados	3
3.2	Preparação dos dados	5
3.2.1	Remoção de duplicados	5
3.2.2	Sanitização e engenharia de atributos	5
3.2.3	Imputação de <i>missing values</i>	6
3.2.4	Tratamento de <i>outliers</i>	6
3.2.5	Seleção de variáveis e separação de ramos	6
3.3	Modelação	7
3.3.1	<i>Clustering</i> de perfis acústicos	7
3.3.2	Classificação de género musical	10
3.3.3	Regressão da popularidade	12
3.4	Avaliação	13
4	Resultados e análise crítica	13
4.1	Leitura crítica do <i>clustering</i>	13
4.2	Leitura crítica da classificação de género	14
4.3	Relação entre <i>clustering</i> e classificação	14
4.4	Leitura crítica da regressão	15
4.5	Limitações	15
	Parte II — Tarefa 2: Dataset Atribuído (Consumo Energético)	16
5	Definição da metodologia	16
6	Exploração	16
7	Transformação	16
8	Modelação e Avaliação	17
8.1	Decision Tree	18
8.2	Random Forest	18
8.3	Gradient Boosted Trees	18
8.4	Multilayer Perceptron	19
8.5	Keras	20

9 Conclusão	22
9.1 Síntese dos resultados do dataset atribuído	22
9.2 Síntese global do trabalho	23
A Anexo A — Workflows da Tarefa 1 (Spotify Tracks)	24
B Anexo B — Figuras complementares da Tarefa 1	27
C Anexo C — Tabelas da Tarefa 1	29
D Anexo D — Workflows da Tarefa 2 (Consumo Energético)	30
E Anexo E — Tabelas da Tarefa 2	33

1 Introdução

O presente relatório documenta o trabalho desenvolvido no âmbito da unidade curricular de Aprendizagem e Decisão Inteligentes, correspondente à componente prática de grupo. O objetivo central consistiu em conceber, implementar e avaliar modelos de aprendizagem automática sobre dois conjuntos de dados distintos, recorrendo à plataforma KNIME como ambiente de desenvolvimento e experimentação.

O trabalho estrutura-se em duas tarefas independentes, conforme definido no enunciado. A **Tarefa 1** incide sobre um **dataset de grupo** — um conjunto de dados sobre faixas musicais do Spotify, gerado sinteticamente para simular um cenário realista de preparação de dados. Nesta tarefa, a metodologia adotada foi o **CRISP-DM**, aplicando-se técnicas de *clustering* para identificação de perfis acústicos, modelos de classificação para previsão do género musical e modelos de regressão para previsão da popularidade das faixas. A **Tarefa 2** é dedicada a um **dataset atribuído** — um conjunto de dados sobre consumo energético e variáveis meteorológicas, distribuído pela equipa docente através do Blackboard. Nesta tarefa, seguiu-se a metodologia **SEMMA**, desenvolvendo-se modelos de classificação com base nas relações entre variáveis climáticas e os padrões de consumo registados.

Cada tarefa é descrita em parte própria do relatório, com a respetiva fundamentação metodológica, descrição das fases de análise e modelação, e discussão crítica dos resultados.

Parte I — Tarefa 1: Dataset de Grupo (Spotify Tracks)

2 Domínio e objetivos

2.1 Domínio analisado

O dataset `spotify_tracks.csv` foi gerado por um *script* Python especificamente concebido para simular um conjunto de dados realista sobre faixas musicais da plataforma Spotify. O ficheiro contém **100 500 registos** e **21 colunas**, cobrindo o período temporal de **2000 a 2024** e **20 géneros** musicais distintos. Cada registo representa uma faixa, caracterizada por atributos de identificação, variáveis de contexto editorial e um conjunto de *features* acústicas extraídas da API do Spotify.

O domínio analisado é o da análise musical e do consumo digital. As *features* acústicas — como `danceability`, `energy`, `loudness` e `acousticness` — capturam propriedades sonoras objetivas, enquanto variáveis como `genre`, `release_year` e `explicit` fornecem contexto editorial. A coluna `popularity` representa uma pontuação sintética entre 0 e 100 associada à visibilidade relativa de cada faixa.

O gerador não produziu um *dataset* limpo; pelo contrário, introduziu deliberadamente problemas de qualidade comuns em cenários reais de análise aplicada. Foram injetados *missing values*, *outliers* numéricos em várias colunas e 500 linhas duplicadas. Existe ainda um ficheiro auxiliar, `spotify_tracks_errors_log.csv`, que regista a localização exata de cada erro introduzido e serviu como referência para validação das etapas de preparação.

Tabela 1: Estrutura do dataset Spotify Tracks

Coluna	Descrição
<code>track_id</code>	identificador sintético único da faixa
<code>track_name</code>	título da faixa
<code>artist_name</code>	nome do artista principal
<code>album_name</code>	título do álbum
<code>release_year</code>	ano de lançamento entre 2000 e 2024
<code>genre</code>	rótulo do género musical principal
<code>explicit</code>	indicador booleano de conteúdo explícito
<code>popularity</code>	pontuação sintética de popularidade entre 0 e 100, usada em tarefas de ranking e na regressão
<code>danceability</code>	grau em que a faixa é ritmicamente adequada para dançar
<code>energy</code>	intensidade e nível de atividade percecionados
<code>loudness</code>	volume global em dB
<code>speechiness</code>	intensidade de conteúdo falado
<code>acousticness</code>	confiança de que a faixa é acústica
<code>instrumentalness</code>	confiança de que a faixa tem pouco ou nenhum conteúdo vocal

Coluna	Descrição
<code>liveness</code>	probabilidade de presença de audiência ao vivo
<code>valence</code>	positividade musical
<code>tempo</code>	BPM estimado
<code>key</code>	tonalidade codificada entre 0 e 11
<code>mode</code>	1 para maior, 0 para menor
<code>time_signature</code>	número estimado de batidas por compasso
<code>duration_ms</code>	duração da faixa em milissegundos

2.2 Objetivos

Os objetivos definidos para esta tarefa foram:

- garantir uma preparação robusta dos dados, com tratamento de duplicados, *missing values*, *outliers* e criação de atributos derivados;
- identificar perfis acústicos através de *k-means* e interpretar a composição dos *clusters* obtidos;
- treinar modelos de classificação para prever o género musical (**genre**, 20 classes) a partir das *features* acústicas, relacionando esta abordagem supervisionada com a segmentação não supervisionada do *clustering*;
- treinar modelos de regressão para prever **popularity** a partir das variáveis acústicas, de contexto e do género musical;
- produzir uma análise crítica dos resultados, com foco na relação entre *clustering* e classificação, na contribuição do género para a regressão e no desempenho preditivo das três vertentes.

3 Metodologia CRISP-DM

A metodologia CRISP-DM (*Cross-Industry Standard Process for Data Mining*) serviu como referência estrutural para a Tarefa 1, mas foi aplicada apenas nas fases relevantes para o trabalho realizado: compreensão dos dados, preparação dos dados, modelação e avaliação. A etapa de compreensão de negócio foi intencionalmente omitida por se tratar de um problema académico já formulado no enunciado, e o processo não avançou para *deployment*, uma vez que o objetivo era analisar e comparar modelos em ambiente experimental.

3.1 Compreensão dos dados

O workflow desta fase encontra-se documentado no Anexo, Figura A.1.

O bloco de *Data Understanding* do *workflow* foi organizado para cobrir as dimensões essenciais da qualidade dos dados. A leitura inicial, através do nó **CSV Reader**, confirmou a importação de 100 500 linhas e 21 colunas, com as *features* de áudio corretamente interpretadas como numéricas.

O nó **Statistics** foi configurado para 12 colunas numéricas — `release_year`, `popularity`, `duration_ms`, `danceability`, `energy`, `loudness`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `valence` e `tempo` — e revelou, de forma quantitativa, os problemas centrais do dataset. As *features* acústicas apresentam cerca de 3000 *missing values* por coluna (por exemplo: 3013 em `danceability`, 3021 em `speechiness` e 3026 em `tempo`). Em paralelo, várias colunas limitadas teoricamente ao intervalo $[0, 1]$ exibem violações claras de domínio, como `danceability` com mínimo -0.299 e máximo 1.497 , `energy` com mínimo -0.298 e máximo 1.498 , e `valence` com mínimo -0.300 e máximo 1.497 . Detetaram-se ainda extremos artificiais em `loudness` (até -89.95 dB) e `tempo` (até 419.95 BPM), consistentes com a injeção deliberada de *outliers* prevista no gerador.

A componente categórica foi examinada com cinco nós **Value Counter**, cobrindo `genre` (20 géneros confirmados), `explicit`, `mode`, `key` e `time_signature`. A exploração univariada assentou em 12 histogramas e 7 *boxplots*, que documentaram as distribuições das variáveis contínuas.



Figura 1: Distribuições de Popularity e Danceability (exemplos de histogramas exploratórios)

As relações bivariadas foram analisadas com 4 gráficos de dispersão — pares `energy-loudness`, `energy-acousticness`, `danceability-valence` e `release_year-popularity` — e uma matriz de correlação linear sobre 15 colunas, visualizada em Heatmap (Figura 2).

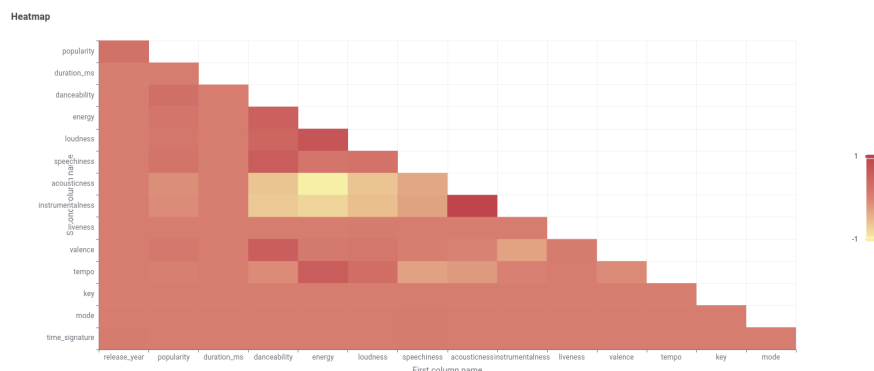


Figura 2: Matriz de correlação linear sobre as principais *features* numéricas do dataset

A deteção de duplicados foi delegada para a fase de preparação, onde o nó **Duplicate Row Filter** os remove. Na fase de compreensão, a presença de padrões suspeitos nas estatísticas (como valores idênticos repetidos em várias colunas) serviu apenas como indicador indireto da necessidade de uma limpeza posterior.

Em síntese, esta fase demonstrou que o *dataset* continha, de forma inequívoca, os quatro tipos de problemas que a preparação subsequente teria de resolver: *missing values*, violações de domínio, *outliers* e duplicados. A Tabela 2 resume a evidência recolhida.

Tabela 2: Problemas de qualidade evidenciados na fase de compreensão dos dados

Problema	Evidência no workflow	Consequência
<i>Missing values</i>	Cerca de 3 000 missings por <i>feature</i> acústica; p.ex. 3 013 em <i>danceability</i> , 3 021 em <i>speechiness</i> , 3 026 em <i>tempo</i>	Imputação em Missing Value
Valores fora de domínio	<i>danceability</i> , <i>energy</i> e <i>valence</i> com mínimos negativos e máximos acima de 1; <i>popularity</i> validada em [0, 100]	Sanitização em Expression
Extremos artificiais	<i>tempo</i> até 419.95, <i>loudness</i> até -89.95 dB, <i>duration_ms</i> com extremos fora do perfil central	Tratamento em Numeric Outliers
Duplicados	Statistics evidencia padrões repetidos; tratamento delegado para a preparação	Remoção em Duplicate Row Filter

3.2 Preparação dos dados

O workflow da preparação encontra-se no Anexo, Figura A.2.

A preparação dos dados foi implementada como um pipeline sequencial com cinco etapas principais, executadas antes da separação entre os ramos de *clustering*, classificação e regressão. Esta preparação comum garante que as três vertentes partem exatamente da mesma base limpa e comparável, divergindo apenas quando os objetivos analíticos passam a exigir conjuntos de atributos diferentes.

3.2.1 Remoção de duplicados

A primeira operação do pipeline é o nó **Duplicate Row Filter**, que compara todas as colunas exceto **track_id**, remove as linhas repetidas e conserva a primeira ocorrência. O resultado confirma a passagem de 100 500 para 100 000 linhas, eliminando os 500 duplicados.

3.2.2 Sanitização e engenharia de atributos

O nó **Expression** concentra num único ponto treze transformações que, num *workflow* menos compacto, estariam dispersas por múltiplos nós. Onze delas são validações de domínio que convertem valores fora dos limites esperados para *missing* (*popularity* [0, 100]; sete *features* de áudio limitadas a [0, 1]; *key* [0, 11]; *mode* $\in \{0, 1\}$; *time_signature* $\in \{3, 4, 5, 6, 7\}$), uma é a codificação binária de **explicit** (True $\rightarrow$ 1, False $\rightarrow$ 0) e uma cria o atributo derivado **duration_min** (= *duration_ms* / 60 000), mais interpretável do que milissegundos. Embora algumas destas *features* nunca violem os respetivos domínios no *dataset* original, as expressões foram aplicadas por precaução, garantindo robustez independentemente dos valores recebidos.

Esta etapa é metodologicamente relevante porque, em vez de eliminar precocemente as linhas problemáticas, opta por converter violações de domínio em *missing values*, delegando a decisão para o nó seguinte.

3.2.3 Imputação de *missing values*

O nó `Missing Value` recebe tanto os *missing values* originais do CSV como os gerados pelo `Expression`. A estratégia de imputação adotada não é uniforme, refletindo uma escolha ponderada por coluna:

- **Média:** `danceability`, `valence`. Estas duas variáveis, depois da sanitização do domínio $[0, 1]$, mantêm distribuições relativamente estáveis e sem caudas extremas comparáveis às de `tempo` ou `loudness`. A média preserva o centro global e introduz menos distorção quando o objetivo é manter a escala contínua original destas medidas.
- **Mediana:** `energy`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `loudness`, `tempo`, `duration_ms`. Nestas colunas existem distribuições mais assimétricas ou mais sensíveis a extremos, pelo que a mediana é mais robusta e evita deslocar artificialmente a massa central por influência de valores muito altos ou muito baixos.
- **Remover linha:** `popularity`, caso a sanitização anterior o converta em *missing*. Nesta fase ainda não se entrou no ramo de regressão, mas já se sabe que a coluna será a variável a prever nesse ramo; por isso, imputá-la introduziria observações com resposta artificial e contaminaria a avaliação posterior.

O resultado executado confirma a passagem de 100 000 para **99 353 registos** — os 647 registos eliminados correspondem a linhas em que a coluna `popularity` ficou inválida após sanitização.

3.2.4 Tratamento de *outliers*

Os *outliers* contínuos são tratados no nó `Numeric Outliers`, que intervém sobre três colunas — `duration_ms`, `loudness` e `tempo` — utilizando o critério IQR com fator 3.0 e substituindo os valores extremos pelo limite admissível mais próximo. A escolha destas colunas foi intencional: são variáveis contínuas em escala aberta, nas quais os *boxplots* e as estatísticas resumidas mostraram extremos claramente isolados da massa principal. Pelo contrário, as *features* limitadas a $[0, 1]$ já tinham sido tratadas antes por sanitização de domínio, e colunas discretas como `key`, `mode`, `time_signature` ou `release_year` não beneficiariam de um corte por IQR, porque os seus valores extremos podem ser semanticamente válidos e não correspondem necessariamente a erro.

O fator IQR 3.0 foi preferido ao valor clássico 1.5 por duas razões. Primeiro, o gerador injeta extremos artificiais bastante afastados, pelo que um limiar mais largo continua a capturar os casos realmente anómalos. Segundo, um corte demasiado agressivo em `tempo` ou `duration_ms` arriscaria comprimir caudas legítimas de músicas invulgarmente longas ou rápidas. Assim, o IQR=3.0 funciona aqui como compromisso entre robustez e preservação da variabilidade natural.

3.2.5 Seleção de variáveis e separação de ramos

O nó `Column Filter` reduz o *dataset* à base comum efetivamente usada pelos ramos de modelação, removendo identificadores (`track_id`, `track_name`, `artist_name`, `album_name`), a coluna redundante `duration_ms` (substituída por `duration_min`) e os atributos `key`, `mode` e `time_signature`. Estas três últimas são variáveis discretas codificadas numericamente que não

são usadas na modelação, porque os modelos principais operam sobre *features* acústicas contínuas e a regressão já dispõe de contexto através de `genre` e `release_year`. A coluna `genre` é preservada. O resultado executado apresenta **99 353 linhas e 14 colunas**, a partir das quais se abrem três ramos.

Esta separação não é meramente organizacional; ela reflete necessidades analíticas distintas. O ramo de *clustering* tem de operar sem rótulos para que os grupos emergentes sejam definidos apenas pela semelhança acústica. O ramo de classificação necessita de manter `genre` como variável a prever, mas deve excluir variáveis editoriais que poderiam enviesar a comparação com o *clustering*. Já o ramo de regressão procura explicar `popularity` com o máximo de contexto útil, pelo que conserva `release_year`, `explicit` e a codificação de `genre`. Desta forma, os três ramos partem da mesma preparação, mas cada um recebe exatamente a informação coerente com o seu objetivo.

Em termos operacionais, os três ramos ficam definidos do seguinte modo:

- **Ramo de *clustering*:** reduzido a 10 *features* acústicas normalizadas (Min-Max para $[0, 1]$), excluindo `genre` e `popularity` para que a segmentação seja inteiramente não supervisionada;
- **Ramo de classificação:** um `Column Filter` isola 11 colunas — as 10 *features* acústicas mais `genre` — e a `Table Partitioner` cria uma partição 80/20, estratificada por género e com `seed=1`, partilhada entre a MLP e a *Random Forest*;
- **Ramo de regressão:** o nó `One to Many` codifica `genre` em 20 colunas binárias (*one-hot encoding*), expandindo a tabela para 33 colunas totais. Neste ramo é também criada a variável auxiliar `popularity_class` (Baixa/Media/Alta) para estratificação da partição 80/20, garantindo que treino e teste mantêm proporções equivalentes de popularidade.

Para suportar as visualizações bidimensionais, foi ainda calculado um PCA comum na fase de preparação, sempre sobre as mesmas 10 *features* acústicas normalizadas. As coordenadas resultantes foram depois reutilizadas nos *scatter plots* de *clustering* e de classificação. Assim, os gráficos PCA são comparáveis entre si: a diferença visual entre eles não resulta de projeções diferentes, mas apenas da variável usada para colorir os pontos (*cluster* ou género previsto).

3.3 Modelação

3.3.1 *Clustering* de perfis acústicos

O workflow de *clustering* está documentado no Anexo, Figura A.3.

Método do cotovelo. A seleção do número de *clusters* baseou-se no método do cotovelo. Foram executadas sete experiências de *k-means* para valores de $k \in \{3, 4, 5, 6, 7, 8, 9\}$, com inicialização aleatória, `seed=1` e 99 iterações máximas. Em cada ramo candidato, a soma das distâncias quadradas intra-*cluster* (WCSS) foi calculada manualmente: o nó `Joiner` associa cada ponto ao centróide do respetivo *cluster*, o nó `Math Formula` calcula a distância euclidiana quadrada às 10 coordenadas do centróide, e o nó `GroupBy` agrega os valores. As sete linhas de WCSS foram concatenadas e representadas graficamente (Figura 3).

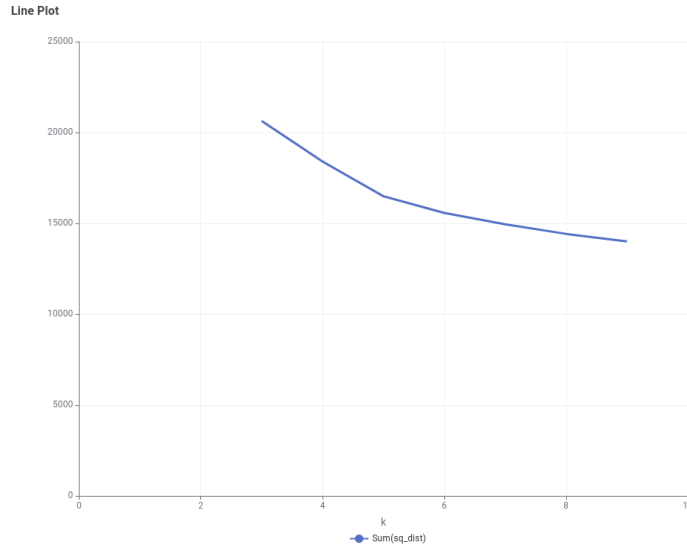


Figura 3: Soma das distâncias quadradas intra-*cluster* (WCSS) para $k \in \{3, \dots, 9\}$ — método do cotovelo

A curva apresenta uma inflexão visível entre $k = 4$ e $k = 6$, com ganhos decrescentes para valores superiores, indicando que a zona de equilíbrio entre compacidade e número de grupos se situa em torno de $k = 5$. Uma escolha muito mais alta, como $k = 20$, seria difícil de justificar: o cotovelo já não sugere ganhos estruturais relevantes depois de 5 ou 6 grupos, a *silhouette* começa a degradar-se a partir daí, e a divisão em 20 grupos tenderia a fragmentar um espaço acústico que já é sobreposto. Em vez de produzir perfis mais claros, isso geraria *clusters* menores, menos estáveis e semanticamente redundantes.

Modelo final. O modelo de *clustering* adotado fixou $k = 5$, coerente com a leitura da curva do cotovelo. O nó **k-Means** produz diretamente a coluna **Cluster**. Os centróides foram transformados para formato longo (**Unpivot**) e visualizados num gráfico de barras agrupado (Figura 4), que permite comparar o perfil acústico médio de cada *cluster* nas 10 *features* normalizadas.

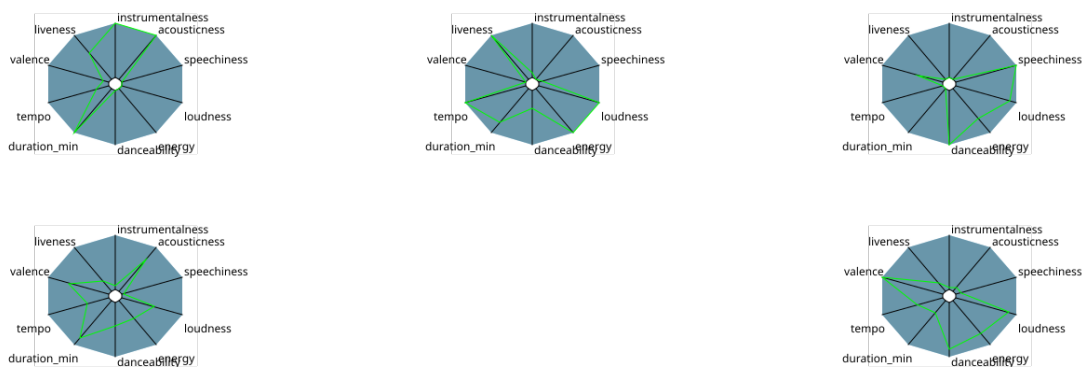


Figura 4: Perfis acústicos médios dos cinco *clusters* ($k = 5$) nas principais *features* normalizadas

Interpretação. A dimensão dos *clusters* foi calculada através de **GroupBy** e **Cross Joiner**, obtendo-se a percentagem relativa de cada grupo. Para interpretação semântica, a coluna **genre** foi reintroduzida por junção com a tabela paralela preservada desde antes da remoção dessa coluna, permitindo construir uma matriz **Cluster** $\times$ **Genre** com **Pivot**. A versão em barras

empilhadas encontra-se no Anexo, Figura B.1.

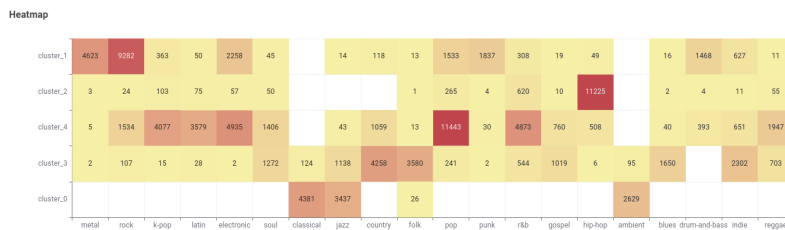


Figura 5: Matriz $Cluster \times Genre$ em *heatmap*, usada para analisar a concentração relativa de géneros em cada grupo

As visualizações mostram que os agrupamentos capturam afinidades estilísticas genuínas, mas sem separações rígidas. O *cluster_0* apresenta o perfil mais acústico e instrumental, concentrando muitas faixas de *classical* e *ambient*. O *cluster_1* reúne faixas mais energéticas, mais altas em volume e, em média, mais rápidas, aproximando-se de zonas do catálogo onde *rock*, *electronic* e subgéneros intensos aparecem com frequência. O *cluster_2* distingue-se por *danceability* e *speechiness* elevadas, formando um grupo próximo de estilos urbanos e rítmicos.

Os *clusters 3 e 4* são importantes precisamente porque mostram que o espaço acústico não se divide apenas em perfis extremos. O *cluster_3* surge como um grupo intermédio e relativamente equilibrado, com valores moderados em várias *features*, funcionando como zona de transição entre perfis mais acústicos e perfis mais energéticos. O *cluster_4* evidencia um perfil próprio, mais melódico e acessível, com combinação de energia moderada, valência relativamente favorável e menor radicalização nas restantes medidas; por isso, tende a agregar géneros populares mais híbridos. A leitura conjunta da Figura 4 mostra que os cinco grupos não são arbitrários: cada um ocupa uma zona acústica reconhecível, ainda que algumas fronteiras sejam difusas.

Validação. A qualidade do agrupamento foi avaliada com o coeficiente de *silhouette*. O método hierárquico (*Euclidean, linkage complete*) operou sobre uma amostra de **5 000 linhas** e produziu um valor médio global de **0.0883**. Para os modelos *k-means*, os valores médios observados foram **0.1802** para $k = 4$, **0.1805** para $k = 5$ e **0.1619** para $k = 6$, calculados sobre amostras estratificadas por *cluster*. A escolha final de $k = 5$ apoia-se na conjugação entre o método do cotovelo e a maior interpretabilidade semântica dos cinco perfis acústicos resultantes.

A projeção PCA bidimensional dos pontos (Figura 6) complementa a análise, confirmando a estrutura de sobreposição sugerida pelos valores de *silhouette*. Esta projeção usa o PCA calculado na preparação dos dados, sobre as mesmas *features* musicais normalizadas usadas nos gráficos da classificação, garantindo comparabilidade visual entre os ramos.

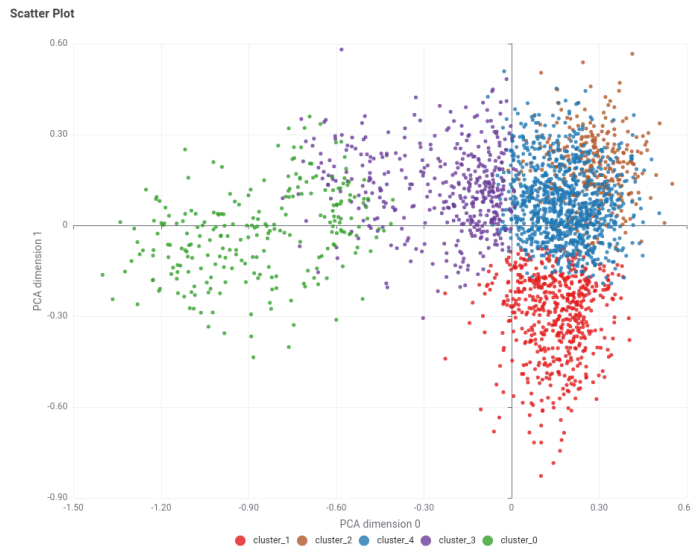


Figura 6: Projeção PCA dos pontos coloridos por *cluster*

3.3.2 Classificação de género musical

O workflow de classificação está no Anexo, Figura A.4.

A novidade face ao ramo de *clustering* é a inclusão de um ramo de classificação supervisionada que utiliza as mesmas 10 *features* acústicas, mas com o objetivo de prever o género musical (20 classes). Este desenho permite contrastar diretamente a aprendizagem não supervisionada (agrupamento por semelhança acústica, sem conhecimento dos rótulos) com a aprendizagem supervisionada (classificação informada pelo género real), medindo até que ponto os atributos de áudio contêm sinal suficiente para discriminar estilos musicais.

Configuração. O ramo de classificação parte da base comum pós-preparação. O nó **Column Filter** isola 11 colunas: as 10 *features* acústicas e o target **genre**. A normalização Min-Max $[0, 1]$ é aplicada através do nó **Normalizer**, garantindo que todas as variáveis contribuem em escala comparável.

Foram treinados dois modelos supervisionados:

- **RProp MLP:** rede neuronal *feedforward* com **duas camadas escondidas** de 32 neurónios, treinada com o algoritmo *RProp* durante 400 iterações (*seed*=1). Usa a partição 80/20 estratificada por género (80 000 exemplos de treino e 20 000 de teste).
- **Random Forest:** *ensemble* de **100 árvores** de decisão, critério Gini, profundidade máxima 20, amostragem de colunas em modo $\sqrt{\cdot}$ (*SquareRoot*), com *seed*=1. Usa a mesma partição 80/20 estratificada por género.

O uso de uma partição partilhada entre os dois classificadores torna a comparação direta metodologicamente mais limpa: MLP e *Random Forest* são avaliadas sobre exatamente o mesmo conjunto de teste. Para a *Random Forest*, foi ainda realizado um estudo de sensibilidade em etapas, começando por árvores pequenas para perceber o efeito da profundidade, fixando depois configurações mais competitivas para testar tamanho mínimo de nó, critério de divisão e número de árvores. A MLP foi refinada em três rondas: primeiro variou-se o número de neurónios numa única camada escondida (4, 8, 16, 32, 64) com 100 iterações; depois, fixado o melhor número de neurónios (32), variou-se o número de iterações (100, 200, 400, 1000); por fim, testou-se

uma arquitetura com duas camadas escondidas (2×32 neurónios, 400 iterações), que produziu o melhor resultado.

Resultados. A avaliação foi realizada através do nó **Scorer**, que calcula a exatidão (*accuracy*), o erro, o número de classificações corretas e incorretas, e o coeficiente Kappa de Cohen. A Tabela 3 sintetiza os resultados obtidos.

Tabela 3: Métricas de classificação — Previsão de género musical (20 classes)

Modelo	Exatidão	Erro	Corretas	Kappa
RProp MLP (80/20, 400 iter, 2 hidden layers 32)	0.6877	0.3123	13 753	0.6610
Random Forest (80/20, 100 árvores, Gini, prof. 20)	0.6838	0.3162	13 675	0.6564

Análise por classe. A matriz de confusão visualizada no *Heatmap* (Figura 7) permite inspecionar a distribuição dos erros por classe, revelando que alguns géneros são classificados quase perfeitamente enquanto outros apresentam erros frequentes.

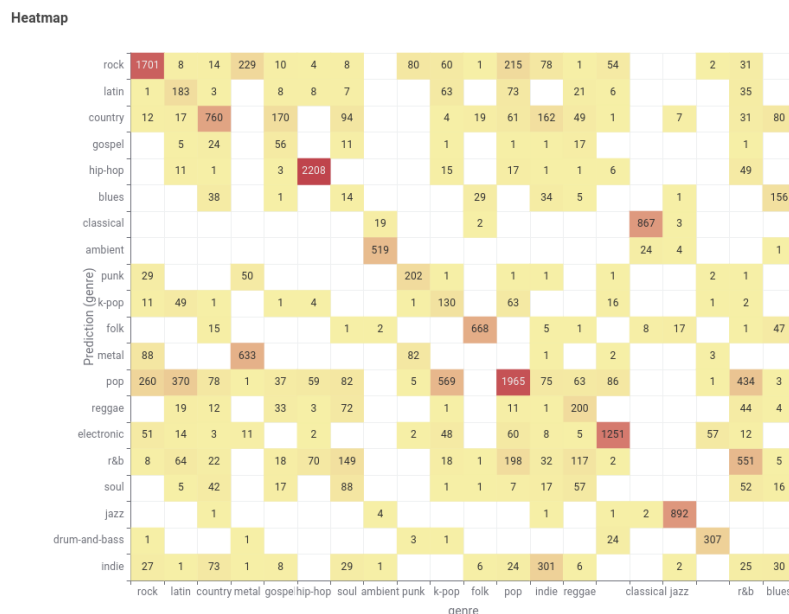


Figura 7: Matriz de confusão — classificação de género via *Random Forest* (20 classes)

A *Random Forest* do ramo principal distingue quase perfeitamente géneros com assinatura acústica mais estável, como *jazz* ($recall \approx 0.990$), *classical* (≈ 0.973), *hip-hop* (≈ 0.951) e *ambient* (≈ 0.951). Em contrapartida, classes como *soul* (≈ 0.293), *k-pop* (≈ 0.422), *r&b* (≈ 0.450) e *latin* (≈ 0.456) revelam separação muito mais fraca, refletindo a maior sobreposição acústica destes géneros.

A projeção PCA das previsões da *Random Forest* (Figura 8) confirma visualmente esta sobreposição: as diferentes classes formam manchas com contornos difusos, com algumas classes mais compactas e outras dispersas. Tal como no *clustering*, o gráfico usa as coordenadas do PCA comum calculado na preparação, pelo que a comparação entre a Figura 6 e as figuras de classificação é feita no mesmo plano geométrico.

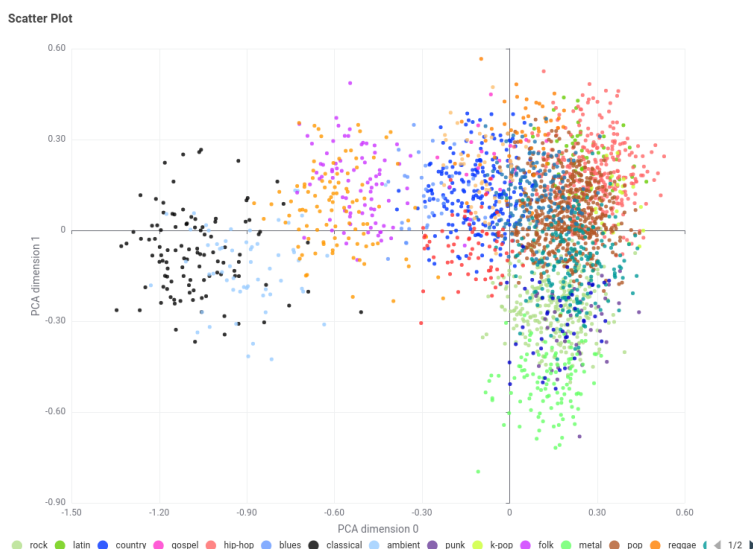


Figura 8: Projeção PCA das previsões da *Random Forest* (pontos coloridos por género previsto)

A projeção PCA das previsões da RProp MLP confirma padrões semelhantes e, por ser redundante face à leitura da *Random Forest*, foi colocada no Anexo (Figura B.2).

Estudo de sensibilidade da *Random Forest*. Foram testadas várias configurações adicionais de *Random Forest*, variando o critério de divisão (Gini vs. *Information Gain Ratio*), a profundidade máxima, o tamanho mínimo de nó e o número de árvores. A metodologia foi incremental: primeiro aumentou-se a profundidade com 100 árvores e tamanho mínimo 5; depois, mantendo profundidade elevada, testaram-se tamanhos mínimos menores; finalmente, com a melhor configuração estrutural, aumentou-se o número de árvores para verificar se a maior estabilidade do *ensemble* melhorava a generalização. Os resultados mostram que a partir das configurações intermédias o desempenho converge para um patamar estreito: 68.16% (gini_16_5_100), 68.38% (gini_20_1_100), 68.36% (gini_20_1_200) e 68.44% (gini_20_1_500). A troca do critério Gini por *Information Gain Ratio* não melhora o desempenho em configurações comparáveis (68.38% vs. 68.20%), indicando que a fronteira de decisão é relativamente robusta à variação do critério de divisão.

A Tabela C.1, no Anexo, resume as configurações mais representativas do estudo.

3.3.3 Regressão da popularidade

O workflow de regressão encontra-se no Anexo, Figura A.5.

Configuração. O ramo de regressão foi expandido para incluir o género musical em *one-hot encoding*. Além dos **12 preditores** originais (`release_year`, `explicit` e as 10 *features* de áudio, incluindo `duration_min`), o nó One to Many codifica os 20 géneros musicais em colunas binárias, totalizando **32 preditores** úteis para os modelos. Esta expansão concretiza a hipótese de que o género musical transporta informação complementar às *features* acústicas para explicar a variabilidade da popularidade. A normalização Min-Max é aprendida no treino e aplicada ao teste, sem fuga de informação.

Foram treinados três modelos: regressão linear múltipla, árvore de regressão simples e *Random Forest*, todos com 32 preditores e variável resposta `popularity`.

Regressão linear múltipla. O modelo linear, treinado com `Linear Regression Learner`,

inclui constante e utiliza todos os preditores disponíveis. As previsões foram avaliadas com **Numeric Scorer**. O diagnóstico visual de resíduos encontra-se no Anexo, Figura B.3.

Árvore de regressão. Para a árvore de regressão, o *workflow* contém um ramo principal com **Simple Regression Tree Learner**, e foram avaliadas configurações exportadas com diferentes níveis de poda/regularização. A afinação foi feita variando primeiro a profundidade para controlar a complexidade global da árvore e, depois, aumentando os mínimos de registos por nó e por nó filho para reduzir divisões demasiado específicas. Entre as experiências, a melhor foi a configuração 5_40_20 (profundidade máxima 5, mínimo de registos por nó 40, tamanho mínimo de nó filho 20). O histograma de resíduos encontra-se no Anexo, Figura B.4.

A Tabela C.2, no Anexo, apresenta o estudo de configurações da árvore de regressão.

Random Forest. O modelo de *Random Forest*, configurado no nó **Random Forest Learner (Regression)**, utiliza *seed*=1, *bootstrap* ativo e amostragem de colunas em modo $\sqrt{\cdot}$ (*SquareRoot*). A configuração final selecionada foi 12_80_1000: profundidade máxima 12, tamanho mínimo do nó 80 e 1000 árvores. O histograma de resíduos encontra-se no Anexo, Figura B.5.

Os resultados adicionais da *Random Forest* de regressão apresentam-se na Tabela C.3, no Anexo. A metodologia de afinação seguiu três passos. Primeiro, fixou-se o número de árvores em 100 para obter testes rápidos e comparáveis. Segundo, com profundidade sem limite, variou-se o tamanho mínimo do nó; valores muito pequenos apresentaram maior tendência para memorizar ruído, enquanto valores maiores suavizaram as previsões. Terceiro, fixado o tamanho mínimo 80, testou-se a profundidade da árvore e, por fim, aumentou-se o número de árvores para 500 e 1000, procurando maior estabilidade sem aumentar o risco de *overfitting*. Embora algumas configurações tenham resultados muito próximos, 12_80_1000 foi escolhida como compromisso final entre erro, estabilidade e controlo da complexidade.

3.4 Avaliação

A avaliação quantitativa da regressão baseou-se em R^2 , MAE e RMSE, calculados via **Numeric Scorer**. A Tabela 4 resume os resultados comparativos dos três modelos com a melhor configuração de cada.

Tabela 4: Métricas de regressão — Spotify Tracks (com género em *one-hot*)

Modelo	R^2	RMSE	MAE
Regressão linear múltipla	0.0459	18.194	13.718
Árvore de regressão (5_40_20)	0.0346	18.301	13.808
<i>Random Forest</i> (12_80_1000)	0.0436	18.215	13.743

4 Resultados e análise crítica

4.1 Leitura crítica do *clustering*

Os resultados de *clustering* mostram que existe estrutura nos dados musicais, mas a separação entre grupos não é nítida. Um coeficiente de *silhouette* de 0.0883 no método hierárquico constitui um sinal de sobreposição relevante entre *clusters*, o que é consistente com a natureza do domínio: diferentes faixas podem partilhar níveis semelhantes de energia, valência e dançabilidade sem que

isso se traduza em grupos radicalmente distintos. Em termos práticos, o *clustering* revela-se mais útil como ferramenta de segmentação exploratória do que como mecanismo de rotulagem rígida.

A análise dos centróides, dos perfis individuais por *cluster*, do *heatmap Cluster × Genre* e da projeção PCA permite uma leitura mais completa dos cinco grupos. O *cluster_0* funciona como polo acústico/instrumental; o *cluster_1* representa o polo energético e mais agressivo; o *cluster_2* capta o perfil mais dançável e mais próximo de conteúdo falado; o *cluster_3* ocupa uma posição intermédia e transversal; e o *cluster_4* agrega um perfil moderado e melodicamente acessível. Esta estrutura faz sentido precisamente porque os cinco grupos equilibram diferenciação com legibilidade. Com menos grupos, perdem-se nuances; com demasiados grupos, como aconteceria numa escolha como $k = 20$, o método passaria a subdividir regiões já sobrepostas sem ganho interpretativo real.

Os valores médios de *silhouette* para *k-means* foram 0.1802 para $k = 4$, 0.1805 para $k = 5$ e 0.1619 para $k = 6$, mostrando que $k = 4$ e $k = 5$ são praticamente equivalentes em qualidade geométrica. A preferência por $k = 5$ resulta, assim, menos de um ganho quantitativo e mais de uma melhor legibilidade semântica dos perfis obtidos. A Figura 6 reforça esta conclusão: os grupos existem, mas sobrepõem-se nas fronteiras, o que justifica simultaneamente a utilidade exploratória do método e os seus limites como mecanismo de separação dura.

4.2 Leitura crítica da classificação de género

Os resultados de classificação demonstram que as 10 *features* acústicas contêm sinal discriminativo substancial para a previsão do género musical. No ramo principal do *workflow*, a *Random Forest* atinge 68.38% de exatidão e a MLP 68.77%, ambas com Kappa em torno de 0.66. Este resultado é expressivo quando comparado com a *baseline* aleatória de 5% (1/20 géneros). Nas experiências suplementares, a melhor *Random Forest* atinge 68.44% e a melhor MLP (2 hidden layers, 32 neurónios, 400 iterações) atinge 68.77%, mostrando que a rede neuronal ligeiramente mais profunda conseguiu extrair um pouco mais de sinal das *features* disponíveis.

A leitura por classe mostra um comportamento muito desigual: géneros com assinatura acústica mais estável (jazz, classical, hip-hop) são classificados quase perfeitamente, enquanto géneros mais híbridos (soul, k-pop, r&b) revelam separação muito mais fraca. Esta assimetria sugere que a dificuldade do problema reside menos na complexidade do classificador e mais na sobreposição intrínseca entre classes no espaço acústico.

4.3 Relação entre *clustering* e classificação

A coexistência dos dois ramos sobre o mesmo espaço de *features* permite uma triangulação metodológica com conclusões convergentes:

1. **Ambas as abordagens encontram sinal.** O *k-means* identifica 5 grupos acústicos não aleatórios (*silhouette* positivo e estável entre 0.1619 e 0.1805). A classificação supervisionada recupera o género com 68.38% de exatidão, muito acima dos 5% do acaso. As *features* acústicas têm, de facto, poder discriminativo.
2. **A separação não é total.** A *silhouette* modesta é a contrapartida não supervisionada da exatidão de classificação de cerca de 68.4% — ambas apontam para uma sobreposição intrínseca no espaço acústico que nenhum modelo consegue eliminar.

3. **O *clustering* descobre agrupamentos que a classificação valida.** A matriz *cluster* $\times$ *género* mostra que géneros próximos no espaço acústico coabitam os mesmos *clusters*, e a matriz de confusão da classificação revela que as trocas entre géneros seguem precisamente este mesmo padrão de afinidade acústica. Esta convergência entre técnica não supervisionada e supervisionada reforça a validade de ambas.
4. **Limite fundamental da representação acústica.** O facto de mesmo as melhores configurações testadas não ultrapassarem 68.77% de exatidão estabelece um limite superior empírico para o que é possível extrair das 10 *features* de áudio. Este limite explica por que razão o *k-means* produz *clusters* com *silhouette* modesta.

4.4 Leitura crítica da regressão

A regressão da popularidade confirma a natureza estruturalmente difícil do target. Mesmo após a inclusão do género por *one-hot encoding*, todos os modelos permanecem com R^2 muito baixos: a regressão linear é a melhor das três abordagens com $R^2 = 0.0459$, seguida de perto pela *Random Forest* afinada com $R^2 = 0.0436$, e pela melhor árvore simples com $R^2 = 0.0346$. Os três modelos operam num intervalo muito estreito de erro (RMSE entre 18.19 e 18.30; MAE entre 13.72 e 13.81), o que mostra que aumentar a flexibilidade do modelo melhora ligeiramente a *Random Forest*, mas não altera materialmente a qualidade preditiva global.

O facto de a regressão linear ainda superar ligeiramente os modelos não lineares sugere que o sinal disponível é simultaneamente fraco e relativamente simples. A afinação da *Random Forest* aproxima o desempenho do modelo linear, mas a diferença residual indica que a componente não linear capturada pelas árvores não é suficiente para ultrapassar a limitação informativa das *features* disponíveis.

Globalmente, a regressão é a vertente onde o contraste entre *clustering*/classificação e popularidade se torna mais visível. As mesmas *features* acústicas que conseguem segmentar perfis sonoros e classificar géneros com qualidade razoável quase não conseguem prever a popularidade. Isto implica que a popularidade depende sobretudo de fatores externos ao espaço acústico: capital simbólico do artista, marketing, exposição em playlists, momento cultural e dinâmica social de consumo.

4.5 Limitações

A principal limitação é a natureza sintética do *dataset* que, embora gerado com distribuições inspiradas em dados reais, não substitui plenamente um catálogo musical autêntico. As relações entre *features* acústicas e popularidade podem ser artificialmente atenuadas ou amplificadas pelo processo de geração, e a distribuição dos géneros (uniforme, 20 géneros com igual representação) não reflete a realidade do mercado musical.

Adicionalmente, os valores de *silhouette* variam com a amostragem (0.0883 em 5 000 registos vs. 0.1619–0.1805 em amostras estratificadas), o que sugere que a estimativa da qualidade do agrupamento é sensível ao tamanho da amostra. As métricas de regressão podem ainda ser influenciadas pelas estratégias de imputação e de tratamento de *outliers* adotadas na preparação.

Parte II — Tarefa 2: Dataset Atribuído (Consumo Energético)

5 Definição da metodologia

A metodologia de análise de dados que será utilizada neste problema é a **SEMMA**: *Sample, Explore, Modify, Model e Assess*. A fase de *Sample* encontra-se já concretizada na forma do dataset atribuído, pelo que o presente relatório se foca nas fases seguintes. Assim, começou-se por realizar a fase de *Explore* ou Exploração, cujo objetivo é perceber potenciais tendências e problemas com os dados, para na fase de *Modificação* se fazerem transformações que visam corrigir os problemas identificados na fase de Exploração. Por fim, aplicam-se as fases de *Modelação* e *Avaliação* para cada um dos 5 modelos desenvolvidos para este problema.

6 Exploração

O metanode de Exploração está documentado no Anexo, Figura [D.2](#).

O primeiro passo da exploração dos dados foi a utilização do nó **Statistics** para verificar diversas estatísticas, destacando-se as seguintes observações:

- **Precipitation** e **rain** apresentam muitos missing values ($\sim 90\%$);
- **Snowfall** tem todos os valores a 0 (mean = min = max = 0);
- **RowID** é redundante;
- Todas as restantes features apresentam uma quantidade semelhante de missing values, cerca de 10%.

De seguida, foi verificada a presença de outliers, recorrendo tanto a **Numeric Outliers** para obter resultados numéricos como ao **Box Plot** para fazer uma exploração mais visual. Desta análise, retirou-se que várias features (como as do vento e as da voltagem) apresentam outliers em quantidade reduzida ($< 2,5\%$) e a **surface_pressure** ligeiramente mais ($\sim 5\%$). No total, são 1 700 as entradas com algum outlier, pelo que a sua remoção não foi considerada, pois perder-se-ia uma boa parte do dataset (cerca de 20%). Portanto, na avaliação dos modelos, os outliers foram ou mantidos ou ajustados para o valor permitido mais próximo, de maneira a avaliar qual o melhor tratamento para cada modelo.

Por fim, foi ainda feita uma análise dos valores das features categóricas com **Value Counter**, para determinar se existem valores fora do padrão. Concluiu-se que as features **Diffuse** e **Direct Radiation** não apresentam valores anormais. Já a target, **Consumptions**, apresenta, em algumas entradas, valores com erros ortográficos (e.g., *Hgh* e *Lw*) ou escritos em português em vez de inglês (e.g., *MedioAlto* vs. *Medium-High Consumption*).

7 Transformação

O metanode de Transformação está documentado no Anexo, Figura [D.3](#).

As primeiras transformações aplicadas aos dados centraram-se nos formatos das features, incluindo a conversão das colunas numéricas que se encontravam em string (**Medium Voltage** e **Total**), que foi realizada fora deste metanode, conforme descrito anteriormente, e a passagem das datas de string para um formato adequado, além da consequente extração dos campos mais relevantes para este problema (hora, dia da semana e mês). A feature original com a data completa foi removida para evitar introduzir ruído. Ainda, foram detectadas 4 linhas nas quais a data era inválida (e.g., 2025-11-32), pelo que essas entradas foram removidas, por ser uma quantidade muito reduzida.

De seguida, no seguimento da exploração realizada, foram removidas algumas features, sendo elas:

- **RowID** (redundante);
- **precipitation** e **rain** (~90% missing values);
- **snowfall** (redundante, mean = min = max = 0).

Para resolver as inconsistências na target detetadas na fase de exploração, foi aplicada uma **Rule Engine** para uniformizar o formato dos valores de **Consumptions**, resultando daqui 5 valores possíveis (Low, MediumLow, Medium, MediumHigh, High).

Por fim, concluiu-se não ser viável remover as entradas com missing values, uma vez que, feito isso, o dataset final fica com 1300 linhas (apenas 15% do total). Portanto, foram avaliadas, para cada modelo, diferentes abordagens para imputar os missing values: média e mediana para os numéricos e moda ou valor fixo “Missing” para as strings. Neste último caso, considerou-se a utilização deste valor fixo porque, desta forma, não se introduzem tendências sintéticas (como com a utilização da moda). Além disso, um missing value pode não ser necessariamente algum erro na recolha dos resultados, podendo ser indicativo do resultado esperado na target.

8 Modelação e Avaliação

É agora altura de desenvolver modelos para o presente problema. Foram concebidos 5 modelos de classificação, com 3 baseados em árvores e 2 redes neuronais. Algumas observações relevantes para as secções seguintes:

1. A partição dos dados para treino e teste foi feita, sempre que possível, com recurso aos nós **X-Partitioner** e **X-Aggregator**, uma vez que o uso de cross-validation, embora mais custoso a nível computacional, permite aproveitar a totalidade do dataset e reduzir as chances do modelo sofrer de overfitting;
2. Os nodos de cross-validation foram configurados com uma random seed, de modo a ser possível obter resultados reproduzíveis e sem variabilidade aleatória;
3. Não foram testadas todas as combinações possíveis de transformações de features e configurações dos modelos, pois é uma quantidade impraticável. Por isso, a abordagem adotada foi: testar um conjunto de abordagens e as suas combinações; tirar daí o melhor resultado; realizar o próximo conjunto de testes com a combinação obtida anteriormente. Isto pode fazer com que não sejam detectadas as melhores combinações para cada modelo, mas torna este processo de refinamento mais exequível.

8.1 Decision Tree

O metanode da Decision Tree encontra-se no Anexo, Figura D.4.

Tal como mencionado na fase de exploração, os tratamentos de missing values testados foram média e mediana para os valores numéricos e, para os categóricos, moda e valor fixo “Missing”. Já os outliers, tal como dito anteriormente, foram mantidos ou ajustados para o valor permitido mais próximo.

As tabelas completas deste estudo encontram-se no Anexo, Tabelas E.1 e E.2.

Os resultados obtidos mostram que o tratamento de missing values e outliers teve impacto reduzido no desempenho da Decision Tree, verificando-se diferenças muito pequenas entre as diferentes estratégias avaliadas (menos de 1%). Em contrapartida, os hiperparâmetros estruturais da árvore tiveram mais influência no desempenho final. Em particular, o aumento do número de records por nó levou a melhorias consistentes da accuracy e do Cohen’s Kappa. Observou-se também que a utilização de pruning não trouxe vantagens relevantes relativamente à ausência de pruning, obtendo resultados ligeiramente inferiores na maioria das configurações, mas reduzindo a variabilidade ao ajustar os outros parâmetros. Quanto à métrica de divisão utilizada, tanto o Gini Index como o Gain Ratio produziram desempenhos semelhantes, embora o Gini Index tenha apresentado, de forma consistente, resultados ligeiramente superiores. O **melhor desempenho** global foi obtido com Gini Index, sem pruning e com mínimo de 12 registos por nó, atingindo uma accuracy de **0.927** e um Cohen’s Kappa de **0.906**.

8.2 Random Forest

O metanode da Random Forest encontra-se no Anexo, Figura D.5.

Aqui foram testadas as mesmas combinações de tratamento de missing values e outliers que na Decision Tree.

As tabelas completas da Random Forest encontram-se no Anexo, Tabelas E.3, E.4 e E.5.

Tal como observado na Decision Tree, as diferentes estratégias de tratamento de missing values e outliers tiveram impacto praticamente nulo no desempenho da Random Forest. Relativamente aos hiperparâmetros da Random Forest, verificou-se que limitar o número de níveis das árvores provocou uma redução consistente do desempenho. Isto sugere que este problema beneficia de árvores mais profundas, capazes de capturar relações mais complexas entre as features. Também o aumento do parâmetro *Minimum child node size* levou a pequenas perdas de desempenho. Os diferentes critérios de divisão testados (Information Gain Ratio, Information Gain e Gini) apresentaram desempenhos muito semelhantes, embora o Information Gain Ratio tenha obtido, de forma consistente, os melhores resultados. Finalmente, a avaliação do número de modelos mostrou que aumentar o número de árvores acima de 100 não produziu melhorias relevantes. Isto indica que a Random Forest atinge rapidamente estabilidade neste problema, sendo que ensembles maiores apenas aumentam o custo computacional sem ganhos significativos de desempenho.

8.3 Gradient Boosted Trees

O metanode do Gradient Boosted Trees encontra-se no Anexo, Figura D.6.

Ao contrário do que foi feito até agora, não foram avaliadas as estratégias de imputação de missing values, uma vez que este modelo dispõe de mecanismos para lidar com este problema.

Sendo assim, foram testados estes métodos juntamente com o tratamento de outliers com as configurações padrão, antes de começar a variar os hiperparâmetros.

O detalhe da avaliação inicial de missing values e outliers está no Anexo, Tabela E.6.

É possível verificar que a configuração com ajuste de outliers tem um resultado igual a não tratar outliers (usando em ambos os casos XGBoost), pelo que se optou por utilizar esta última abordagem, por não introduzir dados artificiais. Na avaliação que se segue, a variação do learning rate foi feita em conjunto com o número de modelos, ou seja, diminuindo a learning rate, aumenta-se o número de modelos. Isto foi feito porque, ao diminuir a learning rate, cada árvore individual contribui menos para o resultado final, pelo que é necessário aumentar o total de árvores para compensar.

As configurações completas de hiperparâmetros estão no Anexo, Tabela E.7.

Os resultados mostram que diferentes combinações de learning rate e número de modelos conduzem praticamente ao mesmo desempenho máximo (accuracy = 0.932 e Cohen's kappa = 0.913), sugerindo que, para este problema, o aumento do número de modelos juntamente com a redução da learning rate não produzem ganhos relevantes. Em contrapartida, observou-se que configurações com maior profundidade das árvores e com row sampling ativo tendem a apresentar desempenhos ligeiramente superiores, indicando que estes parâmetros têm maior influência no resultado final do que a combinação learning rate / número de modelos.

8.4 Multilayer Perceptron

O metanode do Multilayer Perceptron encontra-se no Anexo, Figura D.7.

Para as redes neuronais, uma transformação de dados importante é a normalização (e conseqüente desnormalização), pelo que foram testados 3 métodos diferentes para a normalização dos dados. No caso do método Min-max, usar o intervalo $[0; 1]$ produziu resultados bastante medíocres. Por isso, o intervalo usado foi $[-1; 1]$, que melhorou a performance da rede por centrar os dados em zero. Ainda, o método decimal scaling também produziu resultados pouco satisfatórios (accuracy na ordem dos 60%), pelo que não foi avaliado com o mesmo detalhe que foram os outros.

A outra transformação essencial tem a ver com as features categóricas nominais, que não são adequadas para dar como input a uma rede neuronal. Foram testadas duas formas de as tornar apropriadas ao modelo:

- **numerar:** atribuir valores numéricos sequenciais às categorias. Esta forma de tratamento pode não ser 100% adequada porque introduz na feature uma noção de ordem que pode não ser verdadeira/aplicável. No entanto, para este problema, as features afetadas usam classificações como *None*, *Low*, *Medium* e *High*, pelo que a numeração pode fazer sentido;
- **One to Many:** nó que, para cada categoria de uma feature, cria uma nova feature cujo nome é o nome da original junto com cada valor, e atribui 0 ou 1 conforme o valor da feature naquela entrada. Este método permite tornar features categóricas adequadas para redes neuronais, sem introduzir noções de ordem como a simples numeração.

Um detalhe importante, que difere consoante qual das duas abordagens é utilizada, é quando se faz a normalização. No caso da numeração, a normalização é feita depois, de modo a englobar os novos valores criados. Assim, evita-se que a feature fique com uma escala diferente das restantes. Já no caso de se usar o One to Many, a normalização é feita antes, pois os valores

já estão numa escala consistente (0 ou 1) e assim não se destrói a interpretação binária, que é precisamente o objetivo desta abordagem.

Considerando tudo isto, começou-se por avaliar todas as combinações destas configurações, uma vez que estas são dependentes umas das outras. Por exemplo, os melhores tratamentos de missing values e outliers com Z-score podem ser os piores para Min-max, e portanto a avaliação não estaria correta. O único método de normalização não avaliado com a mesma profundidade que os outros dois foi o Decimal Scaling, porque fez o modelo obter uma accuracy de ~60% com algumas combinações variadas dos restantes parâmetros em avaliação. Assim, foi possível reduzir ligeiramente o espaço de procura (48 para 32 combinações).

O detalhe completo das combinações de pré-processamento avaliadas no MLP encontra-se no Anexo, Tabela E.8.

O detalhe da variação da arquitetura está no Anexo, Tabela E.9.

Os resultados obtidos mostram que o desempenho do Multilayer Perceptron depende mais das transformações aplicadas aos dados do que os modelos anteriores, com ênfase para a normalização e a representação das variáveis categóricas. Dentre os métodos testados, a normalização Z-score apresentou consistentemente melhores resultados do que Min-max e Decimal Scaling. Relativamente às variáveis categóricas, a abordagem One to Many revelou-se superior à simples numeração das categorias neste problema.

Quanto à arquitetura da rede, verificou-se que aumentar o número de iterações melhorou consistentemente os resultados, indicando que o modelo beneficiou de mais tempo de treino. No entanto, o aumento do número de hidden layers e neurónios nem sempre conduziu a melhorias. Neste problema, os resultados sugerem que arquiteturas relativamente simples já conseguem capturar os padrões mais relevantes presentes no dataset. Assim, o aumento da profundidade e do número de neurónios introduziu maior complexidade de otimização sem acrescentar capacidade útil de generalização em relação aos dados de treino, levando em vários casos a uma degradação do desempenho no conjunto de teste.

8.5 Keras

O metanode do Keras encontra-se no Anexo, Figura D.8.

O último modelo avaliado foi mais uma rede neuronal, desta vez usando a biblioteca Keras, que possibilita explorar arquiteturas mais flexíveis e diferentes métodos de otimização. Portanto, as transformações dos dados avaliadas para o MLP não foram reavaliadas, tendo-se optado por manter a melhor combinação encontrada (normalização Z-score, One to Many para as features categóricas, ajuste de outliers, mediana para os missing values numéricos e moda para os missing values categóricos). Portanto, agora o foco passa a ser a definição da arquitetura da rede e do processo de treino.

Antes de prosseguir para a arquitetura da rede, é necessário efetuar algumas transformações nos dados para ser possível aplicar a rede ao problema de classificação. Em particular, é preciso transformar a target `Consumptions`, pois ao contrário do RProp MLP, estas redes neuronais só recebem e devolvem valores numéricos como resultado. Desta forma, à semelhança do que aconteceu com o modelo anterior, foram testadas duas formas de fazer esta transformação:

- **One to Many:** cria-se uma coluna para cada possível valor da feature original, ou seja, cria-se um conjunto de booleanos. Para o output da rede, usam-se 5 neurónios, obtendo-se 5 valores de 0 a 1 que somam 1, ou seja, probabilidades. Depois é só escolher o maior

para se fazer a previsão daquela entrada. Embora este formato de 0s ou 1s já seja aceite pela rede, a noção lógica destes valores é perdida, pelo que esta poderá não ser a melhor abordagem;

- **numerar:** numerar de 1 a 5 os possíveis valores de **Consumptions**. Este tratamento é adequado porque os valores da target já têm uma ordem própria ($\text{Low} < \text{MediumLow} < \text{Medium} < \text{MediumHigh} < \text{High}$), pelo que o significado da numeração continua a fazer sentido para a rede, ao contrário dos booleanos criados com o método anterior. Aqui, a camada de output passa a ter apenas 1 neurónio. O valor gerado pela rede é limitado ao intervalo válido e depois arredondado (e.g., $6.1 \Rightarrow 5$ e $0.2 \Rightarrow 1$), sendo por fim feita a conversão inversa para o valor em string correspondente.

A abordagem baseada na numeração da target pode ser interpretada como uma aproximação híbrida entre classificação e regressão. Em vez de prever diretamente uma classe discreta através de probabilidades independentes, a rede aprende a estimar um valor numérico contínuo associado ao nível de consumo. Desta forma, o modelo passa também a captar a proximidade ordinal entre as classes (por exemplo, errar de *MediumHigh* para *Medium* é menos grave do que errar para *Low*), algo que não acontece naturalmente numa abordagem puramente categórica. No entanto, esta representação exige maior precisão na previsão dos valores numéricos, uma vez que pequenas diferenças no output podem conduzir, após o arredondamento, a classes incorretas.

Como configuração inicial, usou-se uma hidden layer com 16 neurónios e função de ativação ReLU (tal como em todas as futuras hidden layers adicionadas), além da camada de output com 1 ou 5 neurónios, conforme a transformação aplicada à target. Quanto às configurações desta última, no caso do One to Many, a loss function usada é a Categorical Cross Entropy, juntamente com a função de ativação Softmax, que transforma o output nas probabilidades mencionadas anteriormente. Já no caso de se utilizar a numeração, a loss function usada é a Mean Absolute Error e a função de ativação é a Linear, pois não se devem fazer modificações ao output da rede, de maneira a ser possível arredondar o valor produzido. Por fim, o otimizador utilizado em ambas as situações foi o Adam, com os parâmetros padrão.

Depois da configuração inicial, foram testadas outras arquiteturas com mais hidden layers e também neurónios, mantendo o “afunilamento”, ou seja, reduzindo o número de neurónios em cada layer até chegar ao output. Foram utilizados números de neurónios múltiplos de 2 por conveniência experimental e alinhamento com práticas comuns de implementação.

Após ser determinada a melhor arquitetura da rede e método de conversão da target, foram experimentados diferentes números de epochs, valor que se fixou inicialmente em 5 como compromisso entre capacidade de aprendizagem e custo computacional.

Nota: o uso de cross-validation tornou-se computacionalmente inviável com este modelo, cuja duração do treino se começa a tornar excessivamente longa numa máquina casual, especialmente com mais hidden layers e neurónios. Por isso, foi usado um Table Partitioner, fazendo uma partição de 70/30 com uma random seed fixa.

As tabelas completas de arquitetura e epochs do Keras encontram-se no Anexo, Tabelas [E.10](#), [E.11](#) e [E.12](#).

Os resultados obtidos mostram que a estratégia One to Many é significativamente mais adequada para este problema de classificação multiclasse, atingindo desempenhos superiores de forma consistente em praticamente todas as arquiteturas testadas. Além disso, esta abordagem

demonstrou maior estabilidade, apresentando melhorias mais graduais à medida que se aumentava a complexidade da rede.

Por outro lado, a abordagem baseada na numeração da target, embora inicialmente apresente resultados bastante inferiores, revelou uma maior sensibilidade ao aumento da capacidade da rede e do número de epochs, registando melhorias mais acentuadas com arquiteturas mais profundas e mais tempo de treino. Este comportamento pode ser explicado pelo facto de esta estratégia exigir da rede uma aprendizagem mais precisa das relações entre os diferentes níveis de consumo. Por exemplo, uma determinada entrada pode ter Consumption MediumHigh (4) e a rede dar como output 3.4, que será arredondado para 3 (Medium), embora esteja muito perto do resultado correto. Isto fica ainda mais evidente se analisarmos a matriz de confusão dada pelo Scorer, correspondente à arquitetura de 4 layers (128, 64, 32, 16) com 5 epochs:

Tabela 5: Keras — Matriz de confusão (Numerar, 4 layers 128|64|32|16, 5 epochs)

Real \ Previsão	Low	MediumLow	Medium	MediumHigh	High
Low	509	0	49	0	19
MediumLow	81	148	155	15	16
Medium	35	224	331	260	49
MediumHigh	8	124	85	345	87
High	0	7	0	6	67

O efeito referido é mais acentuado para Medium, onde a rede apresenta dificuldades em determinar a “fronteira” entre os diversos níveis médios de consumo. De forma geral, os resultados evidenciam também que as redes neuronais desenvolvidas com Keras oferecem um grau de flexibilidade e configurabilidade substancialmente superior ao dos modelos anteriormente analisados, tornando a escolha da arquitetura, do método de representação dos dados e dos hiperparâmetros fatores decisivos para o desempenho final do modelo. Embora existisse margem para explorar arquiteturas mais complexas e processos de treino mais prolongados, essa exploração tornou-se limitada pelo elevado custo computacional associado ao treino destas redes.

9 Conclusão

9.1 Síntese dos resultados do dataset atribuído

Seguindo a metodologia SEMMA, começou-se por explorar o dataset de forma a identificar problemas de qualidade dos dados, padrões relevantes e possíveis transformações necessárias para a modelação. A análise realizada permitiu detectar valores em falta, outliers, inconsistências na target e features redundantes, conduzindo posteriormente à aplicação de diferentes estratégias de pré-processamento e transformação dos dados para resolver esses problemas.

Após a fase de modificação, foram desenvolvidos e avaliados cinco modelos de classificação distintos. A Tabela 6 sintetiza os melhores resultados obtidos para cada modelo.

Tabela 6: Comparação dos melhores resultados por modelo — Dataset Atribuído

Modelo	Melhor Accuracy	Melhor Cohen's Kappa
Decision Tree	0.927	0.906
Random Forest	0.930	0.910
Gradient Boosted Trees	0.932	0.913
Multilayer Perceptron (RProp)	0.903	0.875
Keras	0.842	0.798

De forma geral, os resultados mostram que os modelos baseados em árvores apresentaram o melhor desempenho neste problema, sobretudo os métodos de ensemble (Random Forest e Gradient Boosted Trees). Estes modelos revelaram também maior robustez relativamente às transformações aplicadas aos dados, sendo pouco sensíveis às diferentes estratégias de tratamento de missing values e outliers.

Por outro lado, as redes neurais demonstraram maior dependência da preparação dos dados. Em particular, a escolha do método de normalização e da representação das variáveis categóricas teve impacto significativo no desempenho obtido. O modelo desenvolvido com Keras destacou-se pela maior flexibilidade e capacidade de adaptação da arquitetura da rede, permitindo explorar diferentes formas de representação da target e diferentes configurações de treino. Ainda assim, os resultados obtidos permaneceram inferiores aos melhores modelos baseados em árvores.

De forma global, conclui-se que este problema é mais adequado a modelos baseados em árvores, capazes de capturar relações complexas entre as variáveis sem exigir um pré-processamento tão sensível como o requerido pelas redes neurais. Importa ainda referir que os resultados do modelo em Keras, ao contrário dos restantes modelos que foram avaliados com cross-validation, foram obtidos com uma simples partição hold-out, devido ao elevado custo computacional associado ao treino da rede neuronal.

9.2 Síntese global do trabalho

O trabalho desenvolvido abrangeu duas tarefas complementares que permitiram explorar diferentes dimensões da aprendizagem automática. A Tarefa 1 demonstrou que features acústicas do Spotify contêm sinal discriminativo genuíno — suficiente para classificar géneros musicais com cerca de 68% de exatidão e para identificar 5 perfis acústicos distintos através de *k-means* — mas insuficiente para prever a popularidade com qualidade razoável ($R^2 < 0.05$). A Tarefa 2 confirmou que os modelos de ensemble baseados em árvores são altamente eficazes para problemas de classificação com features heterogêneas e missing values, atingindo accuracies superiores a 93% no melhor caso.

O uso da plataforma KNIME permitiu uma implementação modular e reproduzível de ambas as tarefas, com workflows claramente estruturados que facilitaram a experimentação sistemática de hiperparâmetros e estratégias de pré-processamento.

A Anexo A — Workflows da Tarefa 1 (Spotify Tracks)

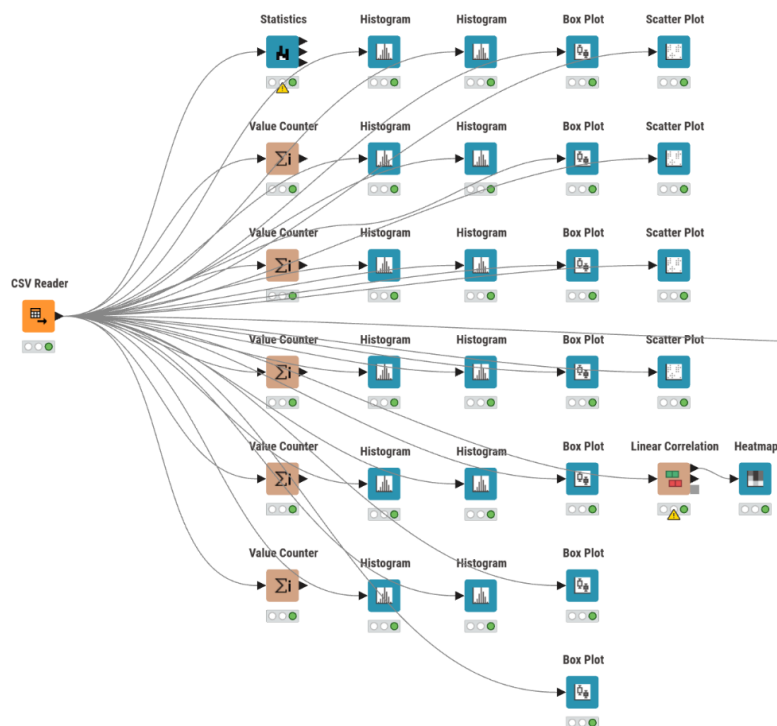


Figura A.1: Workflow da fase de Compreensão dos Dados no KNIME

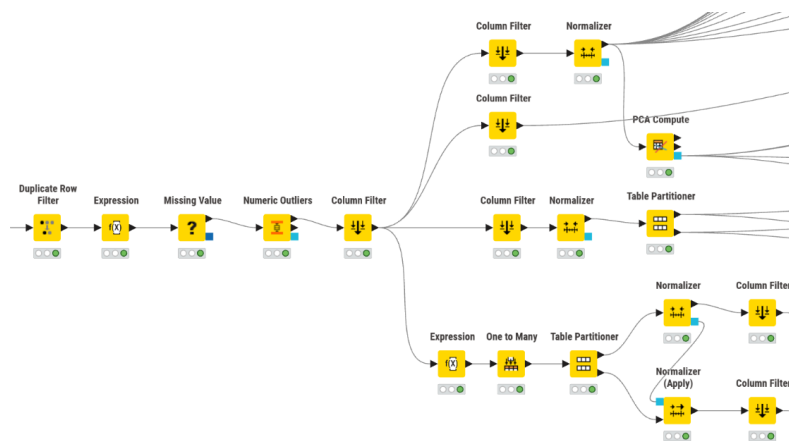


Figura A.2: Workflow da fase de Preparação dos Dados no KNIME

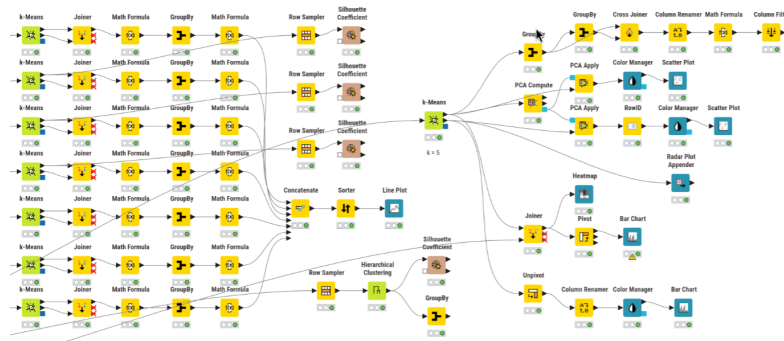
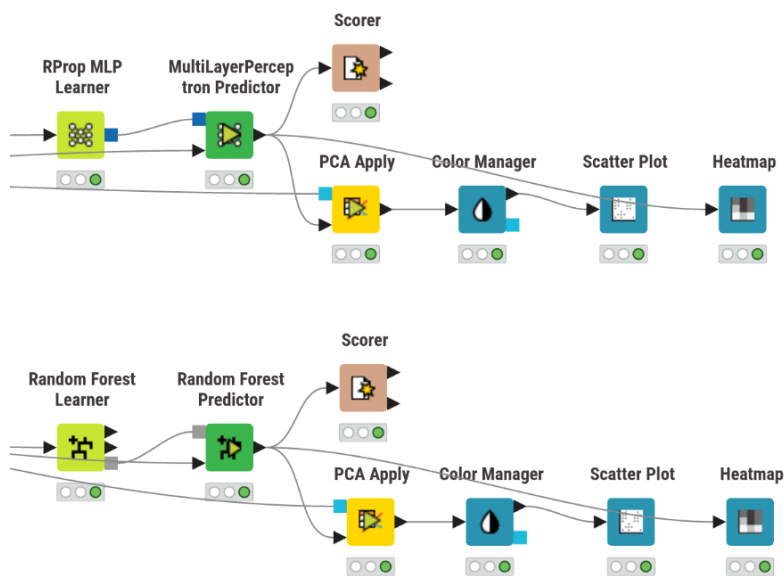
Figura A.3: Workflow da fase de *Clustering* no KNIME

Figura A.4: Workflow da fase de Classificação no KNIME

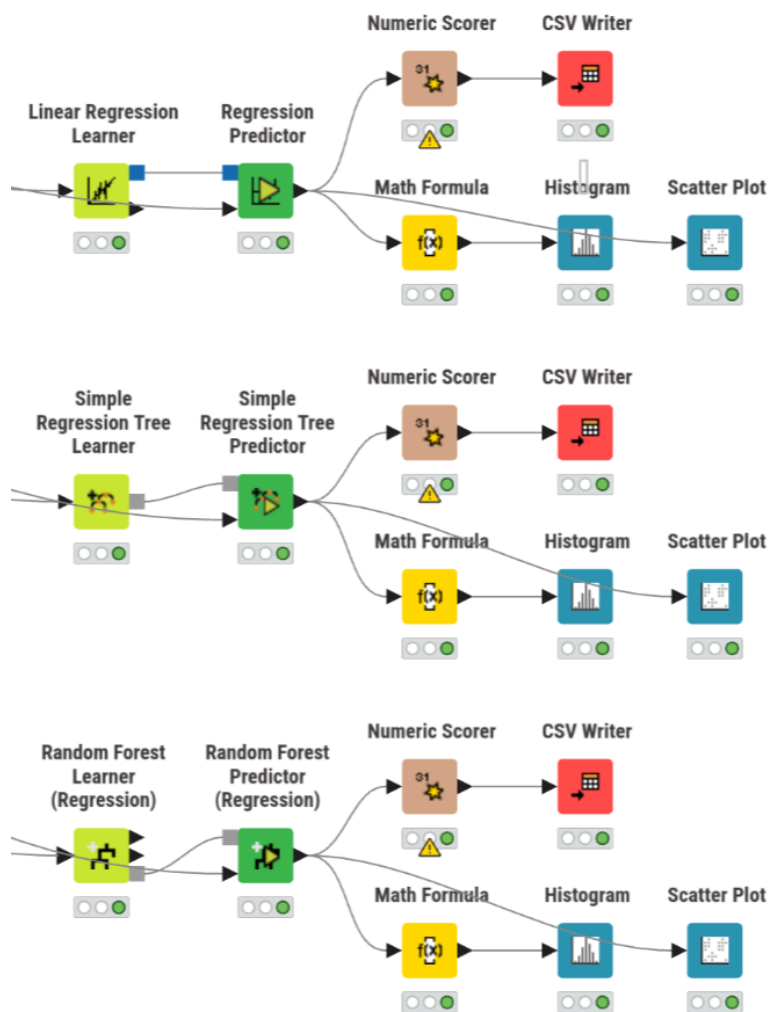


Figura A.5: Workflow da fase de Regressão no KNIME

B Anexo B — Figuras complementares da Tarefa 1

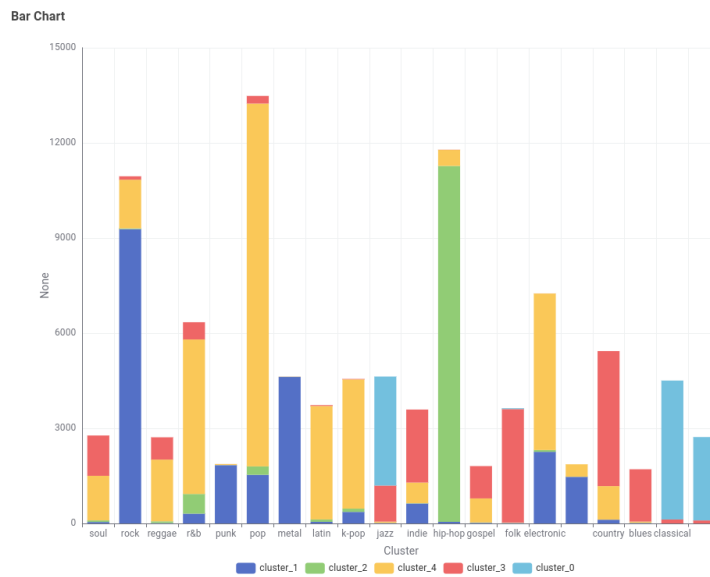


Figura B.1: Distribuição dos géneros musicais por *cluster* (barras empilhadas)

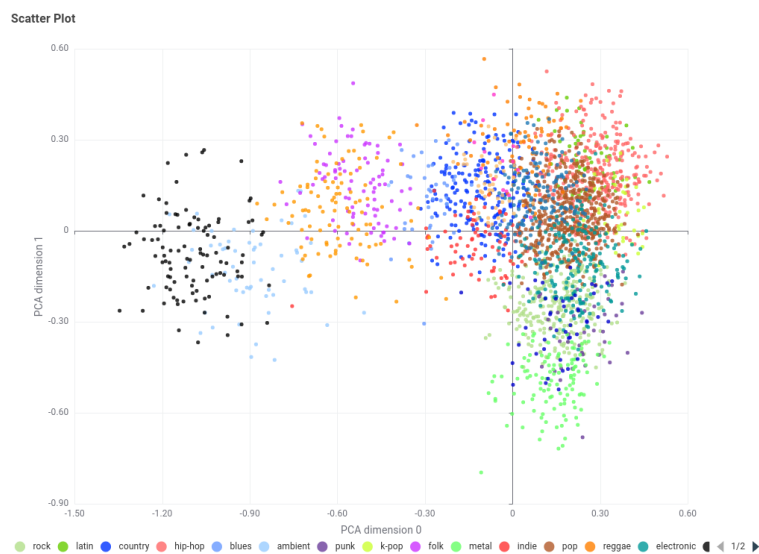


Figura B.2: Projeção PCA das previsões da RProp MLP (pontos coloridos por género previsto)

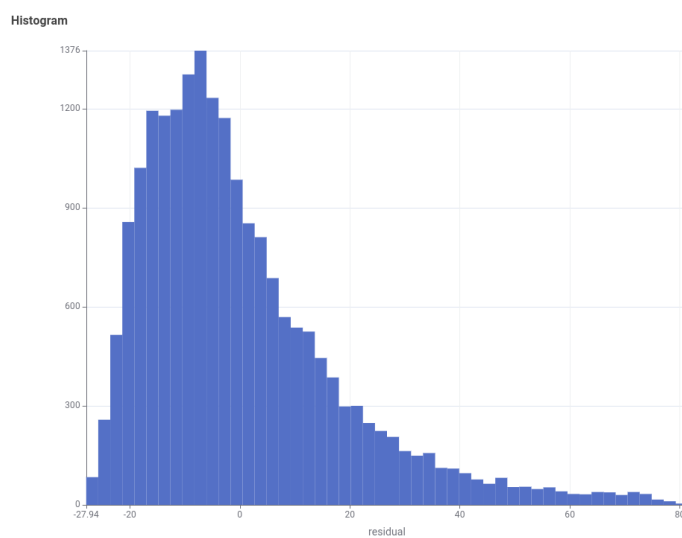


Figura B.3: Diagnóstico do modelo linear: histograma dos resíduos

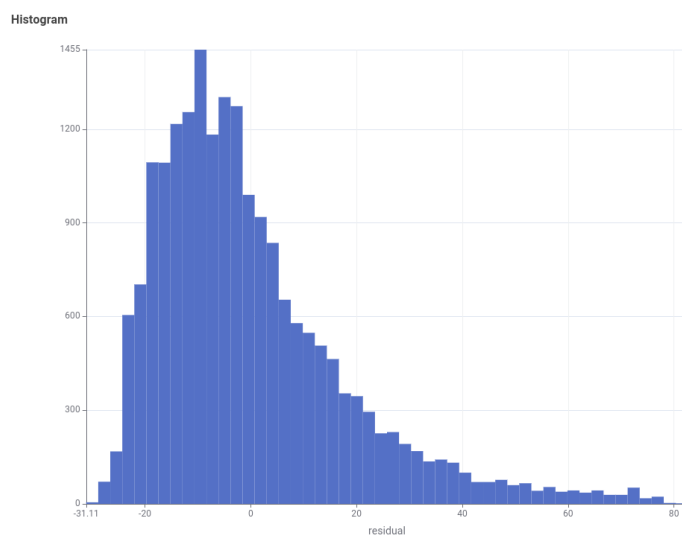


Figura B.4: Diagnóstico da árvore de regressão: histograma dos resíduos

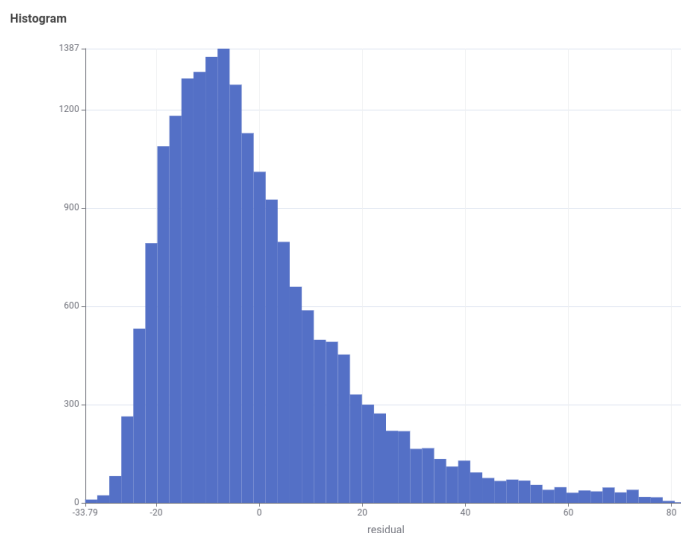


Figura B.5: Diagnóstico da *Random Forest*: histograma dos resíduos

C Anexo C — Tabelas da Tarefa 1

Tabela C.1: Estudo de sensibilidade da *Random Forest* para classificação de género

Configuração	Exatidão	Kappa	Nº árvores
gini_6_5_100	0.5968	0.5229	100
gini_8_5_100	0.6350	0.5632	100
gini_10_5_100	0.6614	0.6120	100
gini_12_5_100	0.6756	0.6397	100
gini_14_5_100	0.6797	0.6443	100
gini_16_5_100	0.6816	0.6480	100
gini_20_5_100	0.6835	0.6509	100
gini_20_1_100	0.6838	0.6564	100
gini_20_1_200	0.6836	0.6561	200
gini_20_1_500	0.6844	0.6571	500
info_gain_ratio_20_1_100	0.6820	0.6543	100

Tabela C.2: Estudo de configurações da árvore de regressão (Profundidade _ MinRegs _ MinFilho)

Configuração	R ²	RMSE	MAE
3_10_5	0.0254	18.411	13.914
5_2_1	-0.0039	18.666	14.047
5_10_5	0.0320	18.355	13.852
5_20_10	0.0335	18.340	13.831
5_40_20	0.0346	18.301	13.808
7_10_5	0.0296	18.379	13.871

Tabela C.3: Estudo de configurações da *Random Forest* de regressão

Configuração	R^2	RMSE	MAE
nolimit_5_100	0.0383	18.265	13.805
nolimit_20_100	0.0423	18.228	13.754
nolimit_80_100	0.0433	18.218	13.745
nolimit_160_100	0.0439	18.213	13.738
8_80_100	0.0418	18.233	13.757
12_80_100	0.0432	18.219	13.744
16_80_100	0.0437	18.215	13.742
12_80_500	0.0436	18.215	13.743
12_80_1000	0.0436	18.215	13.743

D Anexo D — Workflows da Tarefa 2 (Consumo Energético)

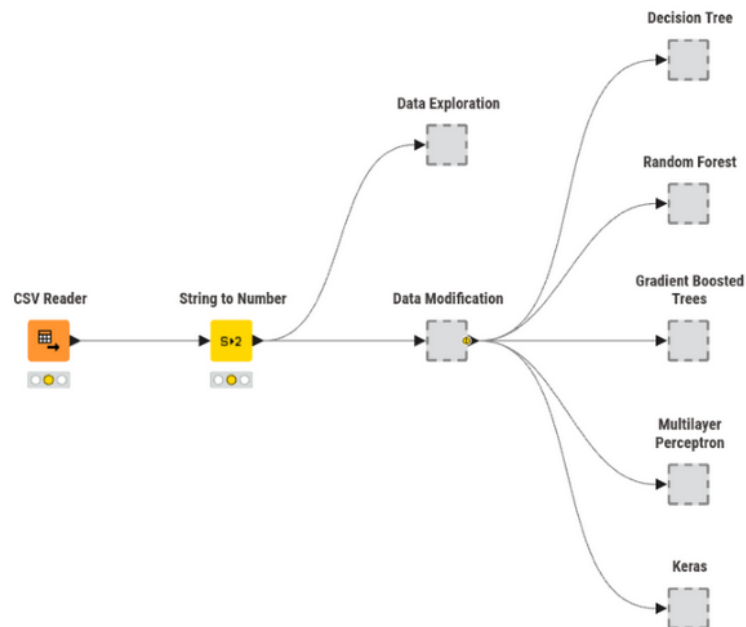


Figura D.1: Visão geral do workflow desenvolvido no KNIME para o dataset atribuído

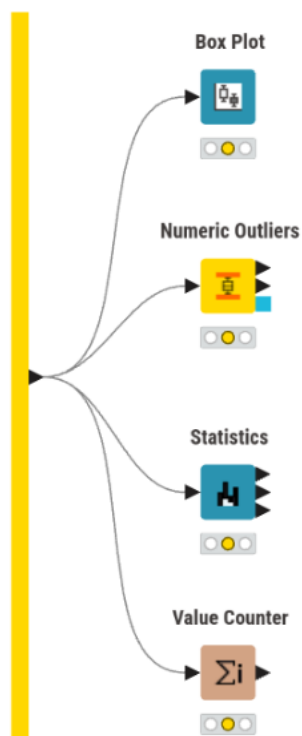


Figura D.2: Metanode de Exploração no KNIME



Figura D.3: Metanode de Transformação no KNIME

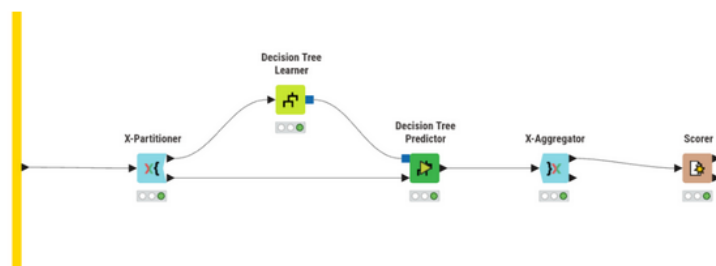


Figura D.4: Metanode do modelo Decision Tree no KNIME

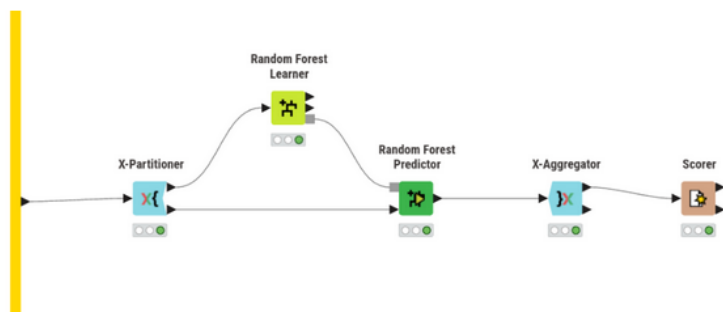


Figura D.5: Metanode do modelo Random Forest no KNIME

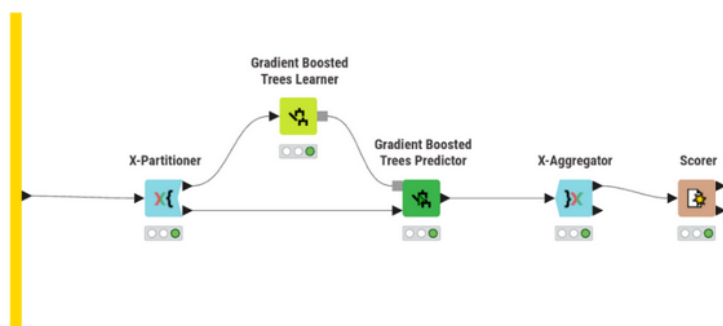


Figura D.6: Metanode do modelo Gradient Boosted Trees no KNIME

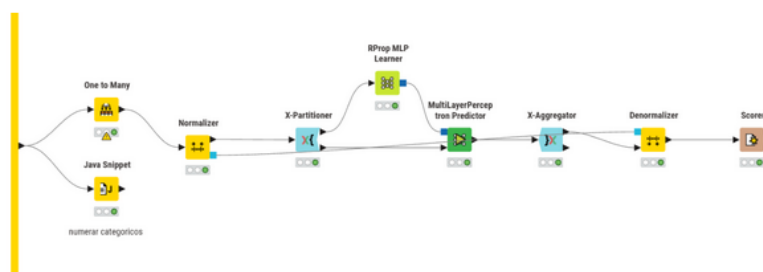


Figura D.7: Metanode do modelo Multilayer Perceptron no KNIME

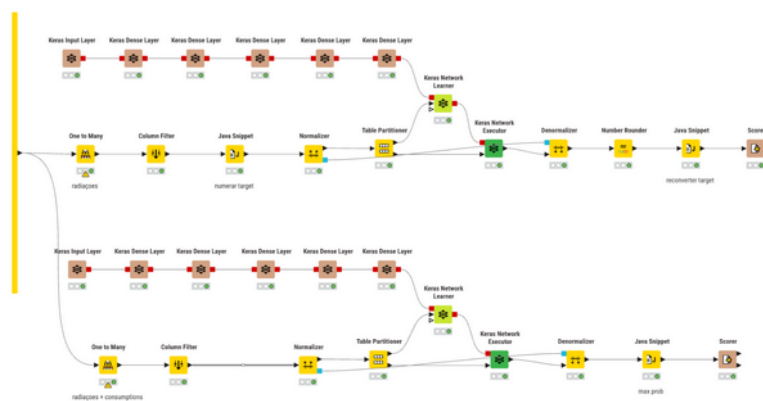


Figura D.8: Metanode do modelo Keras no KNIME

E Anexo E — Tabelas da Tarefa 2

Tabela E.1: Decision Tree — impacto das estratégias de pré-processamento

Outliers	Missing (núm.)	Missing (cat.)	Accuracy	Cohen's Kappa
manter	média	moda	0.884	0.852
manter	média	fixo	0.887	0.855
manter	mediana	moda	0.885	0.853
manter	mediana	fixo	0.888	0.857
ajustar	média	moda	0.885	0.852
ajustar	média	fixo	0.890	0.859
ajustar	mediana	moda	0.885	0.853
ajustar	mediana	fixo	0.889	0.858

Tabela E.2: Decision Tree — variação dos hiperparâmetros estruturais

Quality Measure	Pruning Method	Min recs/nó	Accuracy	Cohen's Kappa
Gini index	No pruning	2	0.890	0.859
Gini index	No pruning	4	0.897	0.869
Gini index	No pruning	8	0.919	0.897
Gini index	No pruning	12	0.927	0.906
Gini index	MDL	2	0.925	0.904
Gini index	MDL	4	0.925	0.904
Gini index	MDL	8	0.925	0.904
Gini index	MDL	12	0.926	0.905
Gain ratio	No pruning	2	0.888	0.857
Gain ratio	No pruning	4	0.904	0.878
Gain ratio	No pruning	8	0.918	0.895
Gain ratio	No pruning	12	0.922	0.901
Gain ratio	MDL	2	0.919	0.897
Gain ratio	MDL	4	0.919	0.897
Gain ratio	MDL	8	0.921	0.899
Gain ratio	MDL	12	0.921	0.899

Tabela E.3: Random Forest — impacto das estratégias de pré-processamento (100 modelos)

Outliers	Missing (núm.)	Missing (cat.)	Accuracy	Cohen's Kappa
manter	média	moda	0.929	0.909
manter	média	fixo	0.929	0.909
manter	mediana	moda	0.930	0.910
manter	mediana	fixo	0.929	0.909
ajustar	média	moda	0.929	0.909
ajustar	média	fixo	0.928	0.908
ajustar	mediana	moda	0.929	0.909
ajustar	mediana	fixo	0.928	0.908

Tabela E.4: Random Forest — variação dos hiperparâmetros (100 modelos, outliers: manter, missing: mediana/moda)

Split criterion	Limit levels	Min child node	Accuracy	Cohen's Kappa
Information Gain Ratio	off	off	0.930	0.910
Information Gain Ratio	off	2	0.929	0.909
Information Gain Ratio	off	4	0.927	0.907
Information Gain Ratio	10	off	0.919	0.896
Information Gain Ratio	10	2	0.919	0.896
Information Gain Ratio	10	4	0.918	0.895
Information Gain	off	off	0.929	0.909
Information Gain	off	2	0.928	0.908
Information Gain	off	4	0.927	0.907
Information Gain	10	off	0.922	0.900
Information Gain	10	2	0.921	0.899
Information Gain	10	4	0.921	0.898
Gini	off	off	0.929	0.909
Gini	off	2	0.928	0.907
Gini	off	4	0.927	0.907
Gini	10	off	0.920	0.898
Gini	10	2	0.920	0.897
Gini	10	4	0.921	0.899

Tabela E.5: Random Forest — variação do número de modelos

Number of models	Accuracy	Cohen's Kappa
50	0.929	0.909
100	0.930	0.910
150	0.930	0.910
200	0.929	0.909
250	0.929	0.909

Tabela E.6: Gradient Boosted Trees — avaliação dos métodos de missing values e outliers

Outliers	Missing values	Accuracy	Cohen's Kappa
manter	XGBoost	0.928	0.908
manter	Surrogate	0.897	0.868
ajustar	XGBoost	0.928	0.908
ajustar	Surrogate	0.896	0.867

Tabela E.7: Gradient Boosted Trees — variação dos hiperparâmetros

Learning Rate	Nº Models	Limit levels	Row Sampling	Accuracy	Kappa
0.1	100	4	off	0.928	0.908
0.1	100	4	0.5	0.929	0.909
0.1	100	4	0.8	0.929	0.909
0.1	100	8	off	0.929	0.909
0.1	100	8	0.5	0.931	0.912
0.1	100	8	0.8	0.932	0.913
0.05	200	4	off	0.928	0.908
0.05	200	4	0.5	0.929	0.909
0.05	200	4	0.8	0.929	0.909
0.05	200	8	off	0.928	0.908
0.05	200	8	0.5	0.932	0.913
0.05	200	8	0.8	0.932	0.912
0.02	400	4	off	0.928	0.908
0.02	400	4	0.5	0.928	0.908
0.02	400	4	0.8	0.928	0.908
0.02	400	8	off	0.930	0.910
0.02	400	8	0.5	0.932	0.913
0.02	400	8	0.8	0.932	0.913

Tabela E.8: MLP — impacto da normalização, encoding e pré-processamento (100 iter., 1 hidden layer, 10 neurónios)

Normalização	Categ.	Outliers	Miss. (núm.)	Miss. (cat.)	Acc.	Kappa
Min-max	numerar	manter	média	moda	0.839	0.793
Min-max	numerar	manter	média	fixo	0.845	0.802
Min-max	numerar	manter	mediana	moda	0.841	0.796
Min-max	numerar	manter	mediana	fixo	0.840	0.795
Min-max	numerar	ajustar	média	moda	0.847	0.803
Min-max	numerar	ajustar	média	fixo	0.838	0.792
Min-max	numerar	ajustar	mediana	moda	0.858	0.818
Min-max	numerar	ajustar	mediana	fixo	0.864	0.826
Min-max	One to Many	manter	média	moda	0.855	0.814
Min-max	One to Many	manter	média	fixo	0.848	0.805
Min-max	One to Many	manter	mediana	moda	0.857	0.817
Min-max	One to Many	manter	mediana	fixo	0.841	0.796
Min-max	One to Many	ajustar	média	moda	0.857	0.817
Min-max	One to Many	ajustar	média	fixo	0.856	0.816
Min-max	One to Many	ajustar	mediana	moda	0.860	0.821
Min-max	One to Many	ajustar	mediana	fixo	0.853	0.812
Z-score	numerar	manter	média	moda	0.862	0.823
Z-score	numerar	manter	média	fixo	0.862	0.823
Z-score	numerar	manter	mediana	moda	0.860	0.820
Z-score	numerar	manter	mediana	fixo	0.860	0.821
Z-score	numerar	ajustar	média	moda	0.864	0.825
Z-score	numerar	ajustar	média	fixo	0.858	0.817
Z-score	numerar	ajustar	mediana	moda	0.867	0.830
Z-score	numerar	ajustar	mediana	fixo	0.856	0.815
Z-score	One to Many	manter	média	moda	0.882	0.849
Z-score	One to Many	manter	média	fixo	0.857	0.817
Z-score	One to Many	manter	mediana	moda	0.876	0.841
Z-score	One to Many	manter	mediana	fixo	0.862	0.823
Z-score	One to Many	ajustar	média	moda	0.878	0.844
Z-score	One to Many	ajustar	média	fixo	0.858	0.819
Z-score	One to Many	ajustar	mediana	moda	0.886	0.854
Z-score	One to Many	ajustar	mediana	fixo	0.858	0.818

Tabela E.9: MLP — variação dos hiperparâmetros de arquitetura (melhor pré-processamento)

Nº iterations	Nº hidden layers	Nº neurons/layer	Accuracy	Cohen's Kappa
100	1	10	0.886	0.854
100	1	20	0.885	0.853
100	1	30	0.882	0.848
100	1	40	0.885	0.853
100	2	10	0.843	0.798
100	2	20	0.870	0.833
100	2	30	0.879	0.845
100	2	40	0.867	0.830
100	3	10	0.829	0.780
100	3	20	0.830	0.782
100	3	30	0.841	0.797
100	3	40	0.856	0.816
200	1	10	0.900	0.873
200	1	20	0.899	0.871
200	1	30	0.896	0.867
200	1	40	0.893	0.863
200	2	10	0.866	0.828
200	2	20	0.889	0.859
200	2	30	0.879	0.845
200	2	40	0.869	0.833
300	1	10	0.903	0.875
300	1	20	0.900	0.872
300	1	30	0.895	0.865
300	1	40	0.888	0.856
300	2	10	0.884	0.851
300	2	20	0.889	0.858
300	2	30	0.877	0.843
300	2	40	0.865	0.828

Tabela E.10: Keras — One to Many (target): variação da arquitetura (5 epochs)

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
1	16	0.576	0.457
2	16, 8	0.639	0.537
2	32, 16	0.724	0.647
3	32, 16, 8	0.752	0.682
3	64, 32, 16	0.777	0.715
4	128, 64, 32, 16	0.779	0.718

Tabela E.11: Keras — Numerar (target): variação da arquitetura (5 epochs)

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
1	16	0.308	0.116
2	16, 8	0.297	0.102
2	32, 16	0.377	0.201
3	32, 16, 8	0.360	0.182
3	64, 32, 16	0.434	0.272
4	128, 64, 32, 16	0.513	0.371

Tabela E.12: Keras — variação do número de epochs (melhor arquitetura por estratégia)

Estratégia	Nº epochs	Accuracy	Cohen's Kappa
One to Many	5	0.779	0.718
One to Many	10	0.803	0.747
One to Many	20	0.842	0.798
Numerar	5	0.513	0.371
Numerar	10	0.639	0.533
Numerar	20	0.684	0.592